

GHULAM ISHAQ KHAN INSTITUTE OF ENGINEERING SCIENCES AND TECHNOLOGY

Faculty of Computer Science & Engineering
Cybersecurity

CY321 — Secure Software Design

PROJECT REPORT

Mahfooz Mobile

A Secure Citizen-Facing System for Reporting and Blocking Stolen Mobile Phones

Submitted to:
Dr. Zubair Ahmad

Submitted by:

Name	Registration Number
Muhammad Abdullah Kaleem	2024326
Muhammad Haris Zafar	2024396
Muhammad Ibrahim Gulzar	2024409
Muhammad Naqi Afaq	2024433

Submission Date: 18-10-2026

Contents

1 Abstract **2**

2 Introduction **2**
2.1 Background . . . 2
2.2 Problem Statement . . . 2
2.3 Objectives . . . 2
2.4 Scope and Assumptions . . . 3

3 System Overview **3**
3.1 Roles and Access Model . . . 3
3.2 Core Features . . . 3

4 System Architecture **3**
4.1 Three-Tier Design . . . 3
4.2 Frontend . . . 3
4.3 Backend and APIs . . . 4
4.4 Database . . . 4

5 Core Functionality **4**
5.1 Registration and CNIC Uniqueness . . . 4
5.2 PTA-Approved IMEI Enforcement . . . 4
5.3 Authentication, Sessions, and Emergency Reporting . . . 4
5.4 Risk-Based Blocking . . . 4
5.5 Theft Report (FIR Draft) Generation . . . 5
5.6 Recovery, Unblock, Disputes, and Oversight . . . 5

6 Security Design **5**
6.1 Authentication and Session Management . . . 5
6.2 Authorization . . . 5
6.3 Data Protection . . . 5
6.4 Input Validation . . . 5
6.5 Prevention of SQL Injection . . . 5
6.6 Prevention of Cross-Site Scripting (XSS) . . . 6
6.7 Prevention of Buffer Overflow . . . 6
6.8 Tamper-Evident Audit Trail . . . 6

7 Testing and Validation **6**

8 Limitations and Residual Risks **7**

9 Conclusion **7**

1 Abstract

Mobile-phone theft is a widespread problem in Pakistan's major cities. The Pakistan Telecommunication Authority (PTA) can blacklist a stolen handset's IMEI nationwide through its DIRBS/LSDS infrastructure, but reporting a theft today is slow and high-friction, often taking up to 24 hours at the exact moment a victim has just lost their primary means of acting quickly. Mahfooz Mobile is a web-based, self-contained registration and blocking authority that mirrors this model on mocked government data. It allows citizens to register their devices, report a theft and have the device blocked quickly through a dedicated memorised Emergency PIN, generate a pre-filled theft report, and recover access without exposing sensitive credentials.

The project was developed as a complete three-tier application: a plain HTML/CSS/JavaScript frontend, a Node.js/Express REST backend, and a PostgreSQL database. Security was treated as a first-class concern throughout. The system enforces role-based access control, encrypts identifying data at rest, maintains a tamper-evident hash-chained audit trail, and restricts device registration to PTA type-approved IMEIs. This report documents the system's design, architecture, functionality, and — in particular — the measures taken to defend against SQL injection, cross-site scripting (XSS), and buffer-overflow attacks.

2 Introduction

2.1 Background

When a mobile phone is stolen, the window in which decisive action matters most is the first few minutes. The device often holds the owner's authentication factors — SMS codes, authenticator apps, and saved passwords — which means that the very tool a victim would normally use to prove their identity is now in the hands of the thief. PTA's DIRBS (Device Identification, Registration and Blocking System) and LSDS can block a stolen IMEI so it is refused service across Pakistani networks, but the citizen-facing reporting process is slow. Mahfooz Mobile was conceived to demonstrate how a national-scale device-blocking authority could be designed securely to make that reporting fast and trustworthy.

2.2 Problem Statement

The central challenge is a security–usability tension: how can a theft victim authenticate quickly to block their own device at the moment their normal credentials may be compromised, while ensuring that a malicious actor cannot abuse the same fast path to block someone else's phone? A secondary challenge is data integrity: ensuring that only legitimate, government-approved devices can be registered against a national identity number, and that the identity data collected is protected at rest and in the user interface.

2.3 Objectives

- Provide a fast, low-friction way for citizens to report and block a stolen device.
- Restrict device registration so that only PTA type-approved IMEIs can be linked to a citizen's CNIC.
- Protect identifying data (CNIC, phone number, email) through encryption at rest and masking in the UI.
- Assess every theft report against a risk model to balance fast protection against malicious blocking.
- Maintain a tamper-evident audit trail of all security-relevant actions.
- Defend the application against common web attacks, specifically SQL injection, XSS, and buffer overflow.

2.4 Scope and Assumptions

Because real NADRA and DIRBS access is not available to a university project, identity records are simulated with a seeded mock dataset, and network-level blocking is represented as a database state together with a conceptual operator API. These are explicit, documented limitations rather than oversights; they are revisited in Section 8. All other functionality — registration, authentication, risk assessment, reporting, recovery, and auditing — is fully implemented against a real relational database.

3 System Overview

3.1 Roles and Access Model

Access is governed by role-based access control (RBAC), assigned on a least-privilege basis and enforced on the server for every request rather than inferred from the user interface. Four roles are defined:

- **Citizen** — the device owner. Can register devices, report and unblock their own devices, and view their own history.
- **Support Agent** — front-line staff who handle disputes and risk-flagged cases. Can review the queue and mark cases, but cannot unilaterally unblock a device.
- **Auditor** — an oversight role with read-only access to the tamper-evident audit log and the ability to verify its integrity.
- **System** — a service identity used for automated internal actions, logged like any other actor.

3.2 Core Features

The implemented system provides the following capabilities, all backed by the PostgreSQL database:

- Account and device registration with national-identity (CNIC) uniqueness enforcement.
- A PTA type-approved IMEI registry; only approved and active IMEIs may be linked to a CNIC.
- Password- and PIN-based authentication with token-based sessions.
- A dedicated Emergency PIN flow for reporting theft, independent of day-to-day login factors.
- A risk engine that scores each report and decides between an automatic block and manual review.
- Automatic generation of a downloadable, pre-filled theft report (FIR draft).
- Emergency PIN recovery via security questions or a one-time email code.
- A staff case queue, an auditor dashboard, a public IMEI status check, disputes, and notifications.

4 System Architecture

4.1 Three-Tier Design

Mahfooz Mobile follows a classic three-tier architecture. The browser renders the user interface and communicates only over HTTP with the application server; the application server holds all business logic and is the sole component that talks to the database; and the database stores and protects the data. This separation is itself a security control: the browser never receives database credentials or direct query access, so sensitive logic and data cannot be reached or tampered with from the client.

4.2 Frontend

The frontend is built with plain HTML, CSS, and JavaScript, with no third-party framework and no build step. It consists of a set of pages (landing, registration, login, dashboard, device management, emergency reporting, case status, recovery, staff and auditor dashboards, and more). A single client module acts as a thin API client: each user action issues a request to the backend and renders the response. Keeping the frontend simple reduces the attack surface, since there is no framework dependency chain to audit and no bundled third-party code executing in the user's browser.

4.3 Backend and APIs

The application server is implemented in Node.js using the Express framework and the standard PostgreSQL driver. It serves the static frontend and exposes a set of REST API endpoints covering registration, login, emergency reporting, device management, recovery, disputes, the staff and auditor functions, and the audit trail. All business rules — credential verification, the risk engine, the approved-IMEI check, and the audit chain — execute on the server, where they cannot be bypassed by a manipulated client.

4.4 Database

The data tier is PostgreSQL. The schema models every entity in the system: the PTA-approved IMEI registry, accounts, recovery questions, devices, theft cases and their timelines, disputes, notifications, one-time passcodes, sessions, duplicate-registration attempts, and the audit log. The database does far more than store rows — it actively enforces security and integrity rules through constraints, foreign keys, triggers, and functions, so that invalid or unauthorised data cannot be persisted even if an application-layer check were missed.

5 Core Functionality

5.1 Registration and CNIC Uniqueness

Registration is a guided, multi-step process that collects the citizen's name, CNIC, phone number, and email; the device's IMEI and details; an account password and a separate Emergency PIN; and five personal recovery questions. The CNIC is enforced as unique at the database level, so a second account cannot claim a CNIC already registered to another account. A duplicate attempt is rejected, recorded, and flagged as a potential fraud signal for the genuine owner. This is treated as a data-integrity constraint, enforced independently of who is performing the registration.

5.2 PTA-Approved IMEI Enforcement

A defining feature of the system is that only PTA type-approved IMEIs may be linked to a citizen's CNIC. The database maintains a registry of approved IMEIs, each with an approval reference and an active/revoked flag. The devices table references this registry through a foreign key, and an additional database trigger refuses any IMEI whose approval has been revoked. As a result, an attempt to register a device with an unknown or de-listed IMEI is rejected by the database itself, and the user is informed that the IMEI is not PTA-approved. Every IMEI is also validated for correct length and Luhn checksum.

5.3 Authentication, Sessions, and Emergency Reporting

Citizens log in with their CNIC or phone number and password. On success, the server issues a session token that the client presents on subsequent requests. The standout flow is emergency reporting: because the victim may not have access to their usual authentication factors, the theft report uses a memorised Emergency PIN. The citizen identifies their account, selects the stolen device from their own IMEI history, provides their phone number, and enters both the Emergency PIN and their account password. The combination of the PIN and password is verified on the server before any block is applied.

5.4 Risk-Based Blocking

Each report is evaluated by a risk engine that scores how strongly ownership is corroborated. It considers the correct Emergency PIN, whether the phone number provided matches the registered number, whether the selected device is a currently active linked device, and whether the report was made from the registered number rather than only the CNIC. A strong match results in a low-risk assessment and an immediate block; a partial match results in a medium-risk block that is additionally sent to staff for manual verification; and a weak match results in a high-risk outcome where no automatic block is applied and the case is held for a manual ownership check first. This graded response protects genuine users quickly while reducing the chance of false or malicious blocking.

5.5 Theft Report (FIR Draft) Generation

When a device is blocked, the system automatically generates a downloadable theft-report document laid out to match the official Initial Information Report (FIR) template under Section 154 of the Criminal Procedure Code. The document contains the reference number, device details and full IMEI, the owner's details, the risk and block status, and a timeline. The citizen's full CNIC is included on this document as a deliberate, documented exception to the masking rule, because law enforcement requires the complete identifier to act. The document is clearly labelled as a draft that supports, but does not replace, an official police filing.

5.6 Recovery, Unblock, Disputes, and Oversight

A citizen who forgets their Emergency PIN can recover it by correctly answering at least three of their five security questions, or by entering a one-time code sent to their registered email. A blocked device can be unblocked by the owner after re-authenticating. Citizens can raise disputes, which — together with risk-flagged theft reviews — populate the Support Agent's case queue. The Auditor can read the full audit trail and verify its integrity, and anyone can check whether a given IMEI is reported stolen through a public lookup.

6 Security Design

6.1 Authentication and Session Management

Passwords and the Emergency PIN are never stored in plaintext or in any reversible form; only salted hashes are kept, and verification is performed by the database. Sessions are represented by random tokens whose hashes are stored server-side, can expire, and are revoked when the password or Emergency PIN changes. Recovery flows issue a short-lived, single-use recovery token only after the user's identity has been proven, so a PIN cannot be reset merely by knowing an account identifier.

6.2 Authorization

Every device- and account-scoped action is checked on the server against the authenticated user's identity and role. A citizen can only act on their own devices, a Support Agent cannot reverse a block on their own authority, and the Auditor's access to the audit log is strictly read-only. These checks are enforced in the API layer and reinforced by least-privilege database roles, so authorization never depends on the client.

6.3 Data Protection

Identifying fields — CNIC, phone number, and email — are encrypted at rest in the database using symmetric encryption, with the encryption key supplied from the environment rather than hard-coded. To still allow uniqueness enforcement and lookup without exposing the plaintext, the system stores a deterministic keyed hash (a 'blind index') of the CNIC and phone number. Low-sensitivity masked forms (for example, showing only the first and last digits of a CNIC) are kept separately for display, so the user interface never needs the plaintext value.

6.4 Input Validation

All write paths validate their input on the server. IMEIs must be fifteen digits and pass the Luhn checksum; CNICs must match the national format; weak or easily guessed Emergency PINs are rejected; and every field is length-bounded. Because validation runs on the server, it cannot be bypassed by editing the page or calling the API directly.

6.5 Prevention of SQL Injection

SQL injection occurs when attacker-supplied text is concatenated into a SQL statement and thereby changes the meaning of the query. Mahfooz Mobile prevents this by using parameterized queries (prepared statements) for every single database operation. User input is passed to the database driver as a separate list of parameter values, referenced in the SQL only through numbered placeholders; it is never

spliced into the query text. The database therefore treats the input strictly as data, never as executable SQL. For example, entering a classic payload such as an always-true condition with a trailing comment into the login field simply produces a username value that matches no record — it cannot alter the query or bypass authentication. A review of the backend confirms that no SQL statement is built by string concatenation with user input, so the injection vector is closed throughout the application.

6.6 Prevention of Cross-Site Scripting (XSS)

Cross-site scripting occurs when attacker-supplied content is rendered as live HTML or script in another user's browser. The system prevents stored and reflected XSS through consistent output encoding: before any user-controlled value — such as a device nickname, account name, dispute text, or recovery question — is placed into the page, it is passed through an encoding function that converts the HTML-significant characters (less-than, greater-than, ampersand, and both quote characters) into their harmless HTML entities. A payload such as an image tag with an onerror handler is therefore stored faithfully but displayed as inert text; the browser never interprets it as markup or executes it. The same encoding is applied when the server generates the FIR document. In addition, the application uses no dangerous dynamic-execution constructs (such as eval, the Function constructor, or document.write), and status messages are inserted using text-only DOM properties, eliminating further injection points.

6.7 Prevention of Buffer Overflow

A buffer overflow is a memory-safety vulnerability in which a program writes past the bounds of a fixed-size memory buffer — a class of bug characteristic of languages with manual memory management such as C and C++. Mahfooz Mobile is implemented entirely in JavaScript, running in the browser and on Node.js, which are memory-safe managed runtimes: there are no manually allocated buffers or pointer arithmetic for an attacker to overflow, so the classic attack does not apply to this stack. The related concern — unbounded input that could cause resource exhaustion or denial of service — is nonetheless addressed in depth. The browser forms impose maximum-length limits on every field; the application server rejects any request body larger than a fixed size limit; and the database columns are strongly typed and length-bounded (for example, the IMEI is a fixed fifteen-character column). Together these bounds ensure that oversized input is truncated or rejected rather than processed.

6.8 Tamper-Evident Audit Trail

Every security-relevant action is written to an append-only audit log. Each entry is linked to the previous one by a cryptographic hash chain, so that altering or deleting any past entry breaks the chain and is immediately detectable. The database permits only insertions into the log — updates and deletions are revoked even for the application's own role — and a verification routine recomputes the chain to confirm its integrity. This gives the Auditor strong, demonstrable evidence of whether the record has been tampered with, supporting non-repudiation of both citizen and staff actions.

7 Testing and Validation

The system was validated end-to-end across all three tiers — the browser interface, the backend APIs, and the PostgreSQL database. A comprehensive automated validation exercised the complete journey beginning with registration: a new citizen was registered through the full interface, and the database was then queried directly to confirm that the account and its device had actually been created, that the CNIC was stored in encrypted form and separately in masked form, and that all five recovery questions were persisted. The new user was then logged out and successfully logged back in, by both CNIC and phone number, proving that the account was durably stored and reusable.

Beyond registration, the validation confirmed correct behaviour for login (including rejection of an incorrect password), the dashboard and device pages, registering additional devices, the full emergency reporting flow at low, medium, and high risk, theft-report generation, device unblocking, disputes, both Emergency-PIN recovery paths, the staff and auditor functions, and the public IMEI check. Negative cases were explicitly tested: duplicate CNICs, non-approved and invalid IMEIs, and weak PINs were all correctly rejected. Security behaviour was also verified — injected script and SQL payloads were neutralised, and the audit chain verified as intact and detected deliberate tampering. All checks passed

with no client-side errors, and the database state was confirmed to match the expected outcome at each step.

8 Limitations and Residual Risks

Consistent with good security practice, the system's limitations are stated explicitly rather than left for a reviewer to discover:

- **Identity is self-reported.** Without live NADRA access, the system can only check a CNIC against its own records; it cannot confirm that a CNIC genuinely belongs to the person registering it.
- **Enforcement is simulated.** A block is a database state and a conceptual operator check, not a real instruction to Pakistani cellular networks, so it does not by itself stop a handset on a live network.
- **Coverage is opt-in.** Only devices registered in the system beforehand are protected.
- **The generated FIR is a reference draft**, not a legally registered police report.
- **Certain controls are represented at a demonstration level:** authenticator (TOTP) verification is simulated, the one-time email code is shown on screen unless an email service is configured, and the encryption keys ship with demo values that would be replaced by a managed vault in production.

None of these affects the correctness of the security mechanisms described in this report; they define the trust boundary within which the system operates and are appropriate for an academic prototype built on mocked government data.

9 Conclusion

Mahfooz Mobile demonstrates how a national-scale device-registration and blocking authority can be designed and built securely, end to end, on mocked government data. The system delivers a fast, low-friction theft report path that respects the real constraints a victim faces, while a graded risk model guards against abuse. It restricts registration to PTA type-approved IMEIs through database-enforced constraints, protects identifying data through encryption and masking, and maintains a tamper-evident audit trail. Crucially, the application is hardened against the principal web-attack classes: SQL injection is eliminated through parameterized queries, cross-site scripting through consistent output encoding, and buffer-overflow concerns are addressed both by the memory-safe runtime and by disciplined input bounds at every tier. The result is a functional, validated prototype that reflects the secure-by-design principles at the heart of the course.